PROJET DATA · BATCH + TEMPS RÉEL

Plateforme de données d'un distributeur d'électricité

Compteurs intelligents au Sénégal — ingestion multi-sources, streaming, entrepôt de données et détection de fraude. Rapport technique et pédagogique.

7 bases de données spécialisées	**25** services conteneurisés (Docker)	**40** tests automatisés (pytest)
~2 850 lignes de Python	**1 000** compteurs simulés, 60 jours	**1,44 M** relevés horaires

Ce rapport explique **ce que fait la plateforme**, **comment elle est construite**, et surtout **pourquoi chaque choix technique a été fait**. Il se lit à deux niveaux : une vue d'ensemble accessible à tous, puis des sections techniques détaillées pour qui veut entrer dans le moteur.

Sommaire

1. Le problème métier
2. Vue d'ensemble en une image
3. Pourquoi 7 bases différentes
4. Les deux flux : batch et temps réel
5. Le parcours d'une donnée
6. L'entrepôt de données (analyse)
7. La détection de fraude (ML)
8. Qualité, tests et choix d'ingénierie
9. Ce qui resterait à faire en production

1. Le problème métier

Un distributeur d'électricité équipe ses clients de **compteurs intelligents** : des appareils qui mesurent la consommation heure par heure et la transmettent automatiquement. Cela ouvre quatre besoins concrets, qui sont le cahier des charges de la plateforme :

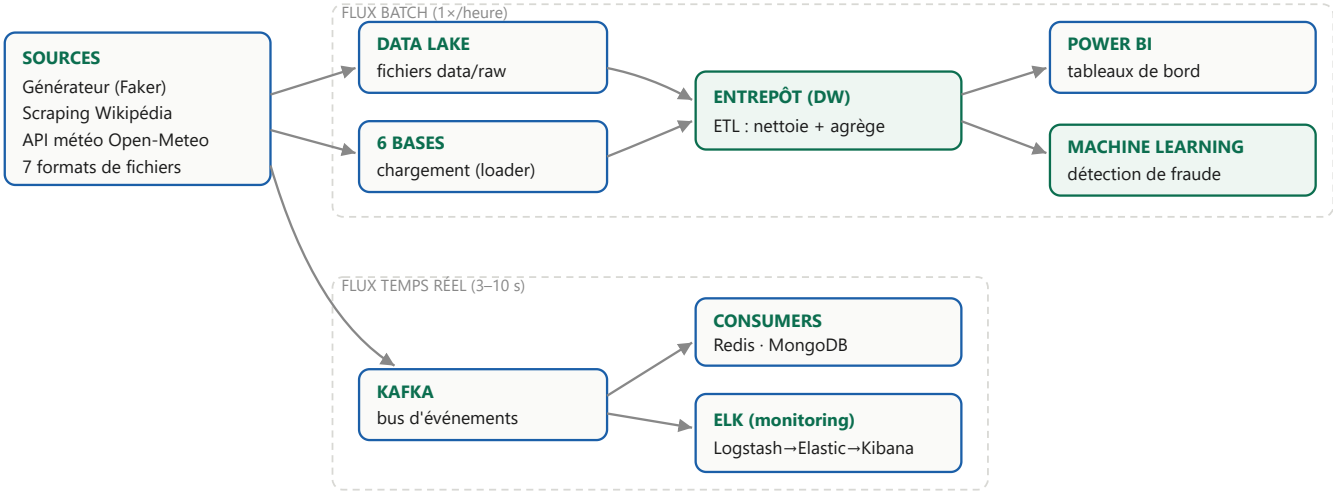
Besoin métier	Réponse de la plateforme
Suivre la consommation **en direct**	Relevés via Kafka → cache Redis → Kibana
Analyser la consommation par zone et par heure	Entrepôt de données + Power BI
Détecter **fraudes et anomalies**	Modèle de Machine Learning sur l'historique
Comprendre l'**impact d'une panne** réseau	Graphe Neo4j : poste → transformateur → compteur

Le domaine est un **distributeur fictif au Sénégal**. Les données sont générées artificiellement mais de façon **réaliste** : vraies régions (codes ISO, population issue de Wikipédia), vraie météo (API Open-Meteo, car la climatisation fait grimper la consommation), tarification par tranches en francs CFA, et même des **défauts de qualité volontaires** dans les données clients (emails manquants, doublons) pour que le nettoyage ait un sens.

> **Pourquoi simuler plutôt qu'utiliser des données réelles ?** Les données de consommation électrique sont sensibles et rarement publiques. Un générateur maîtrisé permet aussi d'**injecter volontairement des fraudes** dont on connaît la vérité — ce qui est indispensable pour entraîner et surtout **évaluer** un modèle de détection.

2. Vue d'ensemble en une image

La plateforme suit un principe simple : **une donnée entre, est transformée, puis exploitée**. Deux vitesses coexistent — le **batch** (par gros paquets) et le **temps réel** (en continu).



Architecture générale : les sources alimentent deux flux (batch et temps réel) qui convergent vers l'exploitation (BI et ML).

3. Pourquoi 7 bases de données différentes

Le réflexe du débutant est de tout mettre dans une seule base (souvent SQL). L'approche « expert » consiste à choisir l'outil selon la **forme** et l'**usage** de chaque donnée. C'est le cœur du projet : chaque base est le bon outil pour un besoin précis.

Base	Contenu	Pourquoi celle-là
PostgreSQL relationnel	Clients, compteurs, réseau (OLTP) + l'entrepôt de données	Données structurées, intégrité garantie par des règles (clés, contraintes)
MySQL relationnel	Facturation : recharges prépayées, factures postpayées	Système de facturation séparé — comme en réalité (souvent un logiciel tiers)
Cassandra colonnes	Historique des relevés (1,44 M de lignes)	Écriture massive continue ; conçue pour les séries temporelles
MongoDB documents	Inspections terrain, alertes	Structure **variable** : une fraude n'a pas les mêmes champs qu'une panne
Neo4j graphe	Topologie du réseau électrique	Répond à « qui dépend de quoi » en suivant les liens, pas en empilant des jointures
Redis clé-valeur	Dernier relevé, consommation live par région	Lecture instantanée du « maintenant » (en mémoire vive)
Elasticsearch recherche	Relevés, alertes, logs (via Logstash)	Recherche et visualisation du flux (monitoring Kibana)

> **L'idée à retenir.** Cassandra et Redis se modélisent **à l'envers** du SQL : on ne part pas des données mais de **la question qu'on posera**. Cassandra range les relevés par compteur et par jour, car la question est « les relevés de ce compteur sur cette période ». Elle ne sait pas répondre à « tous les compteurs d'une région » — et c'est **normal** : cette question d'analyse, c'est le rôle de l'entrepôt.

4. Les deux flux : batch et temps réel

Le flux batch — par gros paquets

Un générateur produit des fichiers dans un **data lake** (zone de stockage brut, organisée par date comme en Big Data). Ces fichiers imitent 7 systèmes sources réels, en 7 formats (CSV, JSON, XML, Excel, YAML…). Un **chargeur** les répartit ensuite dans les 6 bases. C'est lent mais complet : on traite tout l'historique d'un coup.

Le flux temps réel — en continu

Un **producteur** simule les compteurs qui émettent en continu, et publie chaque relevé sur **Kafka**, un « tapis roulant » d'événements. Des **consumers** lisent ce tapis : l'un met à jour le cache Redis (le direct), un autre historise les alertes dans MongoDB, et Logstash indexe tout dans Elasticsearch pour le monitoring.

Le point fort du projet : la cohérence entre les deux. Le temps réel réutilise **exactement le même modèle physique de consommation** que le batch (profils horaires, effet de la température, bruit aléatoire). Un relevé temps réel prolonge donc l'historique sans rupture : mêmes compteurs, même « personnalité » de consommation, index qui continue de monter.

> **Pourquoi Kafka plutôt qu'écrire direct dans les bases ?** Kafka **découple** : le producteur ignore qui lira. Si un lecteur tombe, les messages l'attendent. Et surtout, **plusieurs lecteurs** traitent le même flux pour des usages différents (cache, monitoring, alertes) sans se gêner.

5. Le parcours d'une donnée (exemple concret)

Suivons un relevé du compteur `CPT-10000001` à 20 h, un soir de forte chaleur :

1. **Génération** : le modèle calcule la consommation = base journalière ÷ 24 × profil du soir (pic résidentiel) × effet climatisation × aléa. Résultat : ~0,54 kWh.
2. **Temps réel** : le producteur publie ce relevé sur le topic Kafka `meter-readings`, avec le compteur comme clé (garantit l'ordre).
3. **Cache** : le consumer Redis écrit `compteur:CPT-10000001:dernier` et ajoute la consommation au total live de la région de Dakar.
4. **Monitoring** : Logstash indexe le relevé dans Elasticsearch ; il apparaît dans Kibana, filtrable par région et par heure.
5. **Batch & analyse** : la version historique du même relevé est stockée dans Cassandra, puis l'ETL l'agrège (somme journalière) dans l'entrepôt.
6. **Détection** : si ce compteur se mettait à consommer anormalement peu, le modèle ML le remonterait dans la liste des compteurs à inspecter.

6. L'entrepôt de données (pour l'analyse)

Les bases opérationnelles sont faites pour le **quotidien** (enregistrer un client, un relevé). Elles sont mauvaises pour l'**analyse** (« consommation moyenne par région et par mois ») : données éparpillées, non nettoyées, jointures coûteuses. D'où un **entrepôt de données** séparé : une base unique, propre, et modélisée pour analyser.

Le schéma en étoile

Au centre, les **faits** (les mesures : la consommation). Autour, les **dimensions** (les axes d'analyse : le temps, la région, le client, le compteur). Une question d'analyse devient une simple jointure « faits × dimensions », sans cascade de 6 tables.

Deux niveaux de détail coexistent : les relevés **horaires** (vérité fine) et un **agrégat journalier** par-dessus, qui accélère les analyses mensuelles d'un facteur 24.

L'ETL : là où les défauts sont corrigés

Le programme ETL (Extract-Transform-Load) remplit l'entrepôt. Son moment clé est le **nettoyage** : la source contient 1 010 clients, dont 10 **doublons volontaires** (même personne réinscrite, nom mis en majuscules).

> **Un piège évité par l'analyse réelle des données.** Il existe 59 groupes de clients « homonymes » (même prénom/nom/région) — mais ce ne sont pas tous des doublons ! `Adama Ba` à Dakar apparaît 3 fois : ce sont de **vraies personnes distinctes** (noms très courants au Sénégal). La signature fiable d'un vrai doublon est **même email ET même date d'inscription**. Un nettoyage naïf sur le nom aurait supprimé de vrais clients. Résultat correct : exactement **1 000 clients uniques**.

7. La détection de fraude (Machine Learning)

Objectif : repérer les compteurs anormaux (fraude ou panne) à partir de leur **profil de consommation**, pour **prioriser les inspections terrain** — envoyer les techniciens là où le risque est le plus élevé, sans attendre qu'une panne soit signalée.

Le fil rouge du projet

Le générateur **injecte** trois types d'anomalies dans les relevés ; les **inspections** terrain (MongoDB) en étiquettent une partie (comme en vrai : toutes ne sont pas contrôlées). On a donc de quoi faire un **apprentissage supervisé** : des profils de consommation (les indices) + des étiquettes (anomalie ou non).

Anomalie injectée	Signature dans les données	Indice qui la révèle
Fraude par dérivation (bypass)	consommation qui chute brutalement	ratio 2e moitié / 1re moitié bas
Fraude intermittente	zéros la nuit (compteur shunté)	part d'heures à zéro la nuit
Compteur bloqué	consommation nulle en continu	part d'heures à zéro

Deux précautions d'expert

- **Pas de fuite de données.** Les indices décrivent la **consommation**, jamais le fait d'avoir été inspecté. Les inspections servent uniquement d'étiquettes. Sinon le modèle « tricherait ».
- **Jamais le taux de bonnes réponses (accuracy).** Avec seulement ~4 % d'anomalies, un modèle qui dirait « tout va bien » aurait 96 % de réussite… et serait inutile. On mesure donc le **rappel** (ne pas rater d'anomalie) et la **précision** (ne pas déranger un technicien pour rien).

> **Honnêteté sur les résultats.** Sur ce jeu de données, le modèle atteint des scores quasi parfaits. Ce n'est **pas** représentatif du réel : les anomalies sont injectées de façon très marquée, donc faciles à séparer. Le projet démontre la **chaîne complète et correcte** (indices sans fuite, évaluation adaptée, classement actionnable) ; en production, les fraudes seraient plus subtiles et les scores plus bas.

8. Qualité, tests et choix d'ingénierie

Tests automatisés

40 tests couvrent la logique métier. Un principe a guidé tout le code : **séparer la logique du transport**. Les transformations (nettoyage, agrégation, features ML) sont des fonctions pures, testables **sans aucune base ni réseau**. Les connexions aux bases sont isolées à part. Résultat : on valide le cœur du système en quelques secondes, sans Docker.

Infrastructure reproductible

- **25 services Docker** organisés en **profils** (socle, batch, temps réel, ML, Airflow) : on ne démarre que ce dont on a besoin.
- **Un pilote unique** (`make` / `platform.ps1`) encapsule les scénarios ; un **smoke test** vérifie que chaque base répond.
- **Airflow** orchestre le batch : les tâches lancent les conteneurs existants (le code métier n'est pas dupliqué dans l'orchestrateur).
- Détails anticipés : fins de ligne figées (évite un bug Windows/Linux classique), versions d'images épinglées, secrets hors du code (fichier `.env`).

Décisions documentées (extraits)

> **Un client = un compteur**, garanti par une contrainte d'unicité en base. · **Clés étrangères inter-bases interdites** (MySQL ne peut pas référencer PostgreSQL) : la cohérence est vérifiée par l'ETL. · **Logs en JSON** partout, prêts pour Kibana dès le départ. · **Idempotence** des chargements (relançables sans créer de doublons).

9. Ce qui resterait à faire en production

Ce projet est pédagogique et local. Pour une mise en production, les écarts assumés seraient à combler :

- **Sécurité** : Elasticsearch tourne ici sans authentification (choix de dev) ; il faudrait mots de passe, TLS, et des secrets gérés par un coffre.
- **Haute disponibilité** : chaque base est en un seul nœud ; en production, réplication (Cassandra RF≥3, cluster Kafka multi-broker, Elasticsearch multi-nœuds).
- **ETL incrémental** : aujourd'hui l'entrepôt est reconstruit entièrement à chaque fois (« full refresh ») ; en production on ne traiterait que les nouvelles données.
- **Modèle ML** : réentraînement périodique, suivi de la dérive, et un seuil de décision calibré sur le coût réel d'une inspection ratée vs inutile.
- **Orchestration Docker** : le socket Docker donné à Airflow convient en local, pas en production (préférer un exécuteur Kubernetes).

En l'état, la plateforme couvre **la chaîne complète** : de la génération des données à leur exploitation analytique et prédictive, en passant par le stockage spécialisé, le temps réel et le monitoring — le tout reproductible en quelques commandes.